

AI ASSISTED CODING

Lab-Assignment-20.3

Name: Alluri Chaithra

Htno: 2403A51105

Batch: 06

Task-1:

Prompt:

Generate a simple Python login program using a username and password. First, create it without any validation. Then fix it by adding regex checks for usernames, password length rules, and password hashing. Add 3 test cases for valid, invalid, and short passwords.

Code:

```
1 import re
2 import os
3 import hashlib
4 import hmac
5 import secrets
6 import binascii
7
8 # -----
9 # Naive implementation
10 # -----
11 # stores plaintext passwords (INSECURE)
12 naive_user_db = {} # username -> plaintext password
13
14
15 def naive_register(username: str, password: str) -> bool:
16     if username in naive_user_db:
17         return False
18     naive_user_db[username] = password
19     return True
20
21
22 def naive_login(username: str, password: str) -> bool:
23     stored = naive_user_db.get(username)
24     return stored == password
25
26
27 # -----
28 # Improved implementation
29 # -----
30 USERNAME_REGEX = re.compile(r"^[A-Za-z0-9_]{3,20}$")
```

```

31 MIN_PASSWORD_LEN = 8
32 MAX_PASSWORD_LEN = 64
33 PBKDF2_ITER = 100_000
34 SALT_BYTES = 16
35
36 improved_user_db = {} # username -> {'salt': hex, 'hash': hex}
37
38
39 def is_valid_username(username: str) -> bool:
40     return bool(USERNAME_REGEX.fullmatch(username))
41
42
43 def is_valid_password(password: str) -> bool:
44     return MIN_PASSWORD_LEN <= len(password) <= MAX_PASSWORD_LEN
45
46
47 def hash_password(password: str, salt: bytes = None) -> (str, str):
48     if salt is None:
49         salt = secrets.token_bytes(SALT_BYTES)
50     dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, PBKDF2_ITER)
51     return binascii.hexlify(salt).decode("ascii"), binascii.hexlify(dk).decode("ascii")
52
53
54 def verify_password(
55     stored_salt_hex: str, stored_hash_hex: str, password_attempt: str
56 ) -> bool:
57     salt = binascii.unhexlify(stored_salt_hex.encode("ascii"))
58     dk = hashlib.pbkdf2_hmac(
59         "sha256", password_attempt.encode("utf-8"), salt, PBKDF2_ITER
60     )

```

```
61     attempt_hex = binascii.hexlify(dk).decode("ascii")
62     return hmac.compare_digest(stored_hash_hex, attempt_hex)
63
64
65 def improved_register(username: str, password: str) -> bool:
66     if not is_valid_username(username):
67         # invalid username (bad format)
68         return False
69     if not is_valid_password(password):
70         # password too short/long
71         return False
72     if username in improved_user_db:
73         return False
74     salt_hex, hash_hex = hash_password(password)
75     improved_user_db[username] = {"salt": salt_hex, "hash": hash_hex}
76     return True
77
78
79 def improved_login(username: str, password: str) -> bool:
80     rec = improved_user_db.get(username)
81     if not rec:
82         return False
83     return verify_password(rec["salt"], rec["hash"], password)
84
85
86 # -----
87 # Tests
88 # -----
89 def run_tests():
```

```

90     print("Running tests...")
91
92     # Test 1: valid registration and login
93     user = "valid_user1"
94     pwd = "S3curePassw0rd!"
95     ok = improved_register(user, pwd)
96     login_ok = improved_login(user, pwd)
97     print("Test valid registration/login:", ok and login_ok)
98
99     # Test 2: invalid password (wrong password on login)
100    wrong_login = improved_login(user, "wrongpassword")
101    print("Test invalid (wrong) password login:", not wrong_login)
102
103    # Test 3: short password is rejected at registration
104    short_user = "shorty"
105    short_pwd = "short" # too short (< MIN_PASSWORD_LEN)
106    short_ok = improved_register(short_user, short_pwd)
107    print("Test short password rejected:", not short_ok)
108
109
110    if __name__ == "__main__":
111        # quick demo of naive vs improved
112        print("Naive demo (stores plaintext):")
113        naive_register("alice", "password123")
114        print(" naive login alice/password123 ->", naive_login("alice", "password123"))
115        print("\nImproved demo (regex + length checks + hashing):")
116        print(
117            " register bob with weak password '1234' ->", improved_register("bob", "1234")
118        )

```

```

print(
    " register bob_good with good password ->",
    improved_register("bob_good", "GoodPass123!"),
)
print(
    " login bob_good with correct password ->",
    improved_login("bob_good", "GoodPass123!"),
)
print(" login bob_good with wrong password ->", improved_login("bob_good", "WrongPass123!"))
print()
run_tests()

```


Output:

```
C:\Users\venub\OneDrive\Desktop\AIAC_Lab\Lab-20>python 20_1.py
Naive demo (stores plaintext):
naive login alice/password123 -> True

Improved demo (regex + length checks + hashing):
register bob with weak password '1234' -> False
register bob_good with good password -> True
login bob_good with correct password -> True
login bob_good with wrong password -> False

Running tests...
Test valid registration/login: True
Test invalid (wrong) password login: True
Test short password rejected: True
```

Observation:

The login script did not validate user inputs properly. It accepted any string as a username or password, which could lead to issues like buffer overflow or injection attacks. After analysis, input validation was added using Python's `re` (regular expressions) to ensure only valid characters are accepted. The improved version prevents invalid or malicious input and provides error messages for incorrect formats.

Task-2:

Prompt:

Generate a Python script using SQLite that gets user info by username. First, make it using string concatenation (insecure). Then fix it using parameterized queries to stop SQL injection. Add 3 tests for normal login, injection, and secure query.

Code:

```
1 import sqlite3
2
3 # Setup in-memory database and populate with sample data
4 def setup_db():
5     conn = sqlite3.connect(":memory:")
6     cur = conn.cursor()
7     cur.execute(
8         """
9     CREATE TABLE users (
10         id INTEGER PRIMARY KEY AUTOINCREMENT,
11         username TEXT UNIQUE,
12         display_name TEXT
13     )"""
14     )
15     users = [("alice", "Alice A."), ("bob", "Bob B."), ("charlie", "Charlie C.")]
16     cur.executemany("INSERT INTO users (username, display_name) VALUES (?, ?)"
17     conn.commit()
18     return conn
19
20
21 # Insecure: builds SQL by string concatenation (vulnerable to SQL injection)
22 def insecure_get_user(conn, username):
23     cur = conn.cursor()
24     query = (
25         "SELECT id, username, display_name FROM users WHERE username = '"
26         + username
27         + "'"
28     )
29     # show the built query so the vulnerability is visible
30     print("[INSECURE QUERY]", query)
```

```

31     cur.execute(query)
32     return cur.fetchall()
33
34
35 # Secure: uses parameterized query to avoid SQL injection
36 def secure_get_user(conn, username):
37     cur = conn.cursor()
38     query = "SELECT id, username, display_name FROM users WHERE username = ?"
39     # parameterized execution prevents injected SQL from being evaluated
40     cur.execute(query, (username,))
41     return cur.fetchall()
42
43
44 # Tests
45 def test_normal_login():
46     conn = setup_db()
47     insecure = insecure_get_user(conn, "alice")
48     secure = secure_get_user(conn, "alice")
49     assert len(insecure) == 1 and insecure[0][1] == "alice"
50     assert len(secure) == 1 and secure[0][1] == "alice"
51     print("test_normal_login passed")
52
53
54 def test_injection():
55     conn = setup_db()
56     payload = "x' OR '1'='1"
57     insecure = insecure_get_user(conn, payload)
58     secure = secure_get_user(conn, payload)
59     # Insecure query becomes: WHERE username = 'x' OR '1'='1' -> matches all

```



```

60     assert len(insecure) > 1, "Insecure query should return multiple rows for
61     assert len(secure) == 0, "Secure query should not be vulnerable to injecti
62     print("test_injection passed")
63
64
65     def test_secure_query():
66         conn = setup_db()
67         # another injection attempt that tries to match everyone
68         payload = "alice' OR '1'='1"
69         insecure = insecure_get_user(conn, payload)
70         secure = secure_get_user(conn, payload)
71         assert (
72             len(secure) == 0
73         ), "Secure parameterized query should treat payload as a literal"
74         # insecure likely returns multiple rows; we check secure behavior explicit
75         print("test_secure_query passed")
76
77
78     if __name__ == "__main__":
79         test_normal_login()
80         test_injection()
81         test_secure_query()
82         print("All tests passed.")
83

```

Output:

```

C:\Users\venub\OneDrive\Desktop\AIAC_Lab\Lab-20>python 20_1.py
[INSECURE QUERY] SELECT id, username, display_name FROM users WHERE username = 'alice'
test normal login passed
[INSECURE QUERY] SELECT id, username, display_name FROM users WHERE username = 'x' OR '1'='1'
test_injection passed
[INSECURE QUERY] SELECT id, username, display_name FROM users WHERE username = 'alice' OR '1'='1'
test_secure_query passed
All tests passed.

```

Observation:

The initial Python database script was unsafe because it built SQL queries by directly adding user input into the query string. This method allowed attackers to insert malicious SQL code. The improved version now uses parameterized queries, which separate user input from the SQL command. This

change prevents unauthorized access or modification of database records and ensures secure data handling.

Task-3:

Prompt:

Generate a simple HTML feedback form with JavaScript that shows user input using innerHTML. First, leave it insecure. Then fix it by escaping special characters and adding a Content Security Policy (CSP).

Code:

```
1  <!DOCTYPE html>
2  <html Lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Feedback Form</title>
8  </head>
9
10 <body>
11     <h1>Feedback Form</h1>
12     <form id="feedbackForm">
13         <label for="feedback">Your Feedback:</label><br>
14         <textarea id="feedback" rows="4" cols="50"></textarea><br>
15         <button type="button" onClick="submitFeedback()">Submit</button>
16     </form>
17     <h2>Your Feedback:</h2>
18     <div id="displayFeedback"></div>
19
20     <script>
21         function submitFeedback() {
22             var feedback = document.getElementById('feedback').value;
23             document.getElementById('displayFeedback').innerHTML = feedback;
24         }
25     </script>
26 </body>
27
28 </html>
```

Output:

Feedback Form

Your Feedback:

Good

Submit

Your Feedback:

Good

Observation:

The generated HTML feedback form displayed user input directly on the page, making it vulnerable to XSS attacks. This was fixed by sanitizing user input and escaping HTML special characters before displaying them. A Content Security Policy (CSP) was also suggested to block inline scripts. The updated form now prevents script injection and ensures safe user interaction on the client side.

Task-4:

Prompt:

Generate a code snippet. Identify its security weaknesses, correct them, and explain how the fixes make it safer. Show both the old and new code, and include three tests to prove the secure version works properly.

Code:

```
1  import os
2  import pickle
3  import json
4  import shutil
5  from pathlib import Path
6  import tempfile
7
8
9  # -----
10 # OLD (vulnerable) - Do NOT use
11 # -----
12 def vulnerable_load_config(path):
13     with open(path, "rb") as f:
14         return pickle.load(f)
15
16
17 def vulnerable_backup(src, dst):
18     cmd = f"cp {src} {dst}"
19     return os.system(cmd)
20
21
22 # -----
23 # NEW (secure) implementation
24 # -----
25 def _resolve_and_check(path: Path, base: Path) -> Path:
26     """Ensure path stays inside the allowed base directory."""
27     path_resolved = path.resolve()
28     base_resolved = base.resolve()
29     try:
30         path_resolved.relative_to(base_resolved)
```

```
31     except Exception:
32         raise ValueError(f"Unsafe path access: {path} is outside {base}")
33     return path_resolved
34
35
36 def load_config(path: str, base_dir: str | None = None) -> dict:
37     """Safely load JSON config inside base_dir with validation."""
38     base = Path(base_dir) if base_dir else Path.cwd()
39     p = Path(path)
40
41     try:
42         p = _resolve_and_check(p, base)
43     except ValueError as e:
44         raise ValueError("Path traversal blocked!") from e
45
46     try:
47         with p.open("r", encoding="utf-8") as f:
48             data = json.load(f)
49     except FileNotFoundError:
50         raise ValueError(f"Config file not found: {p}")
51     except json.JSONDecodeError as e:
52         raise ValueError(f"Invalid JSON format: {e}") from e
53
54     if not isinstance(data, dict):
55         raise ValueError("Config must be a JSON object.")
56     return data
57
58
59 def backup_file(src: str, dst: str, base_dir: str | None = None) -> None:
```



```

60     """Securely copy file within base_dir using shutil (no shell)."""
61     base = Path(base_dir) if base_dir else Path.cwd()
62     src_p = _resolve_and_check(Path(src), base)
63     dst_p = _resolve_and_check(Path(dst), base)
64
65     if not src_p.is_file():
66         raise FileNotFoundError(f"Source file not found: {src_p}")
67
68     dst_p.parent.mkdir(parents=True, exist_ok=True)
69     shutil.copy2(src_p, dst_p)
70
71
72 def run_tests():
73     print("Running tests for secure implementation...\n")
74
75     with tempfile.TemporaryDirectory() as tmpdir:
76         base = Path(tmpdir)
77
78         # Test 1: Valid JSON config
79         cfg = base / "config.json"
80         cfg.write_text(json.dumps({"name": "example"}), encoding="utf-8")
81         assert load_config(str(cfg), str(base)) == {"name": "example"}
82         print("Test 1 passed: Valid JSON loaded correctly.")
83
84         # Test 2: Safe file backup
85         src = base / "data.txt"
86         dst = base / "copy" / "data.txt"
87         src.write_text("hello", encoding="utf-8")
88         backup_file(str(src), str(dst), str(base))

```

```

89         assert dst.exists() and dst.read_text() == "hello"
90         print("Test 2 passed: File copied safely without shell commands.")
91
92         # Test 3: Path traversal attempt blocked
93         outside = Path(tempfile.gettempdir()) / "outside_config.json"
94         outside.write_text(json.dumps({"x": 1}), encoding="utf-8")
95
96         try:
97             traversal_path = base / "sub" / ".." / outside.name
98             load_config(str(traversal_path), str(base))
99             raise AssertionError("Test 3 failed: Traversal not blocked")
100         except ValueError:
101             print("Test 3 passed: Path traversal correctly blocked.")
102         finally:
103             try:
104                 outside.unlink()
105             except Exception:
106                 pass
107
108         print("\nAll secure implementation tests passed successfully.\n")
109
110
111     # Expected Output Summary
112
113
114     def print_expected_output():
115         print("-----")
116         print("EXPECTED OUTPUT: Security-Audited Code Summary")
117         print("-----")

```

```
123 - Path traversal with no validation.
124 - Missing error handling.
125
126 2. Fixes in Secure Version:
127 - Replaced pickle with json for safe parsing.
128 - Used shutil.copy2 instead of shell commands.
129 - Added path validation (_resolve_and_check).
130 - Added proper exception handling.
131
132 3. Test Results:
133 Test 1: Valid JSON loaded correctly.
134 Test 2: File copied safely.
135 Test 3: Path traversal blocked.
136 All tests passed.
137
138 4. Final Outcome:
139 The secure version prevents code execution, command injection,
140 and traversal vulnerabilities. It includes strong input validation
141 and safer file handling for improved security.
142 """
143 )
144
145
146 # -----
147 # Run section
148 # -----
149 if __name__ == "__main__":
150     run_tests()
151     print_expected_output()
```

Output:

```
C:\Users\venub\OneDrive\Desktop\AIAC_Lab\Lab-20>python 20_1.py
Running tests for secure implementation...

Test 1 passed: Valid JSON loaded correctly.
Test 2 passed: File copied safely without shell commands.
Test 3 passed: Path traversal correctly blocked.

All secure implementation tests passed successfully.

-----
EXPECTED OUTPUT: Security-Audited Code Summary
-----

1. Vulnerabilities in Old Code:
   - Untrusted deserialization using pickle.
   - Command injection via os.system().
   - Path traversal with no validation.
   - Missing error handling.

2. Fixes in Secure Version:
   - Replaced pickle with json for safe parsing.
   - Used shutil.copy2 instead of shell commands.
   - Added path validation (_resolve_and_check).
   - Added proper exception handling.

3. Test Results:
   Test 1: Valid JSON loaded correctly.
   Test 2: File copied safely.
   Test 3: Path traversal blocked.
   All tests passed.

4. Final Outcome:
   The secure version prevents code execution, command injection,
   and traversal vulnerabilities. It includes strong input validation
   and safer file handling for improved security.
```

Observation:

The code had serious security flaws such as using pickle for deserialization, operating system for command execution and no validation of file paths. These issues could allow malicious code execution or unauthorized file access. The corrected version improved security by replacing pickle with json for file copying, and adding strict path validation checks. These updates ensure the code handles files safely, prevents exploitation, and maintains the integrity of the system.